

Bidirectional and Deep RNN

Named Entity Recognition

- Ali loves **apple**, it keeps him healthy.

Person

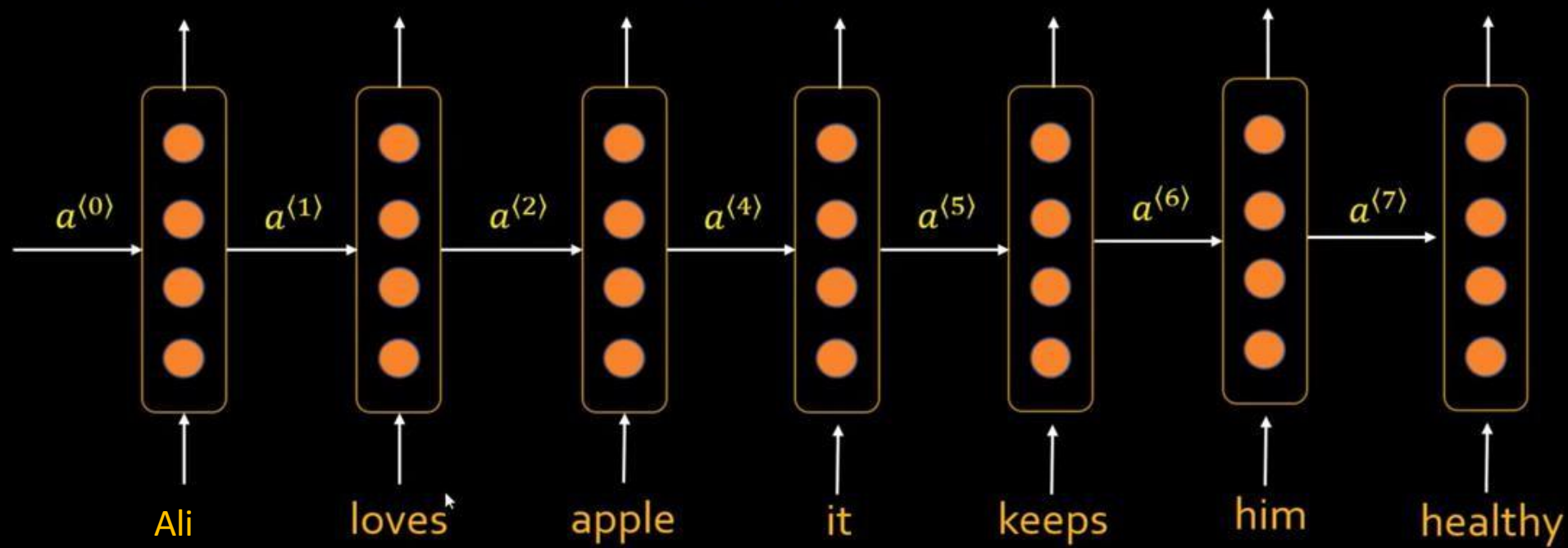
fruit

- Ali loves **apple**, the company produces best electronics.

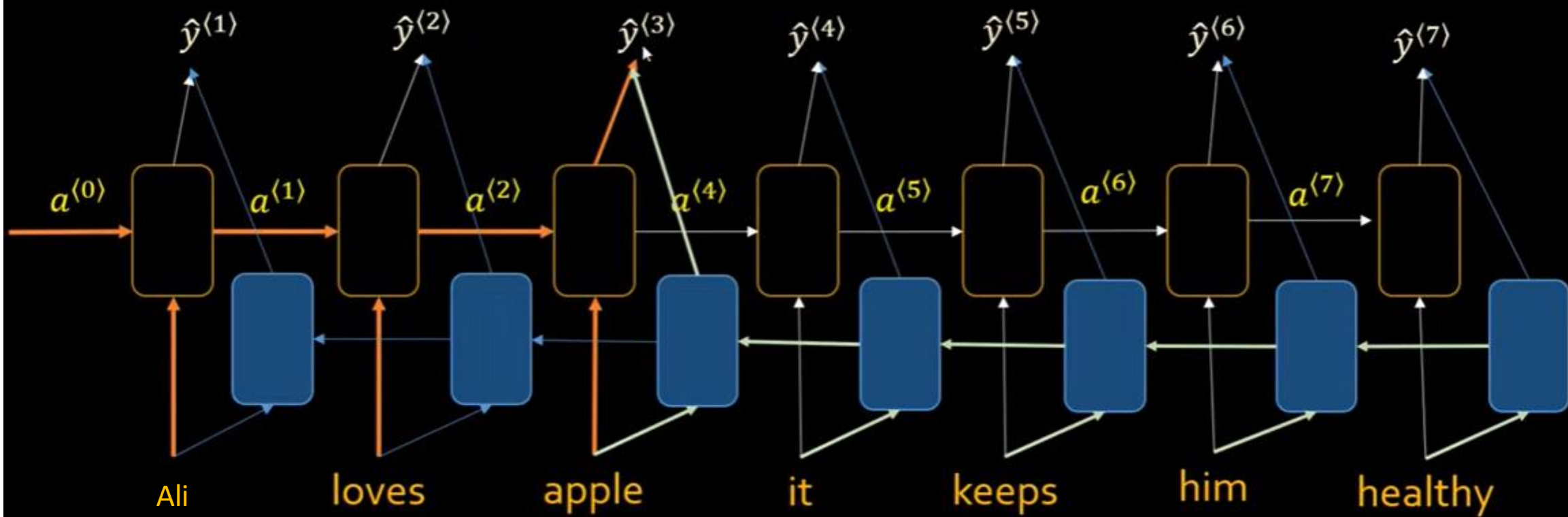
Person

company

fruit or company?



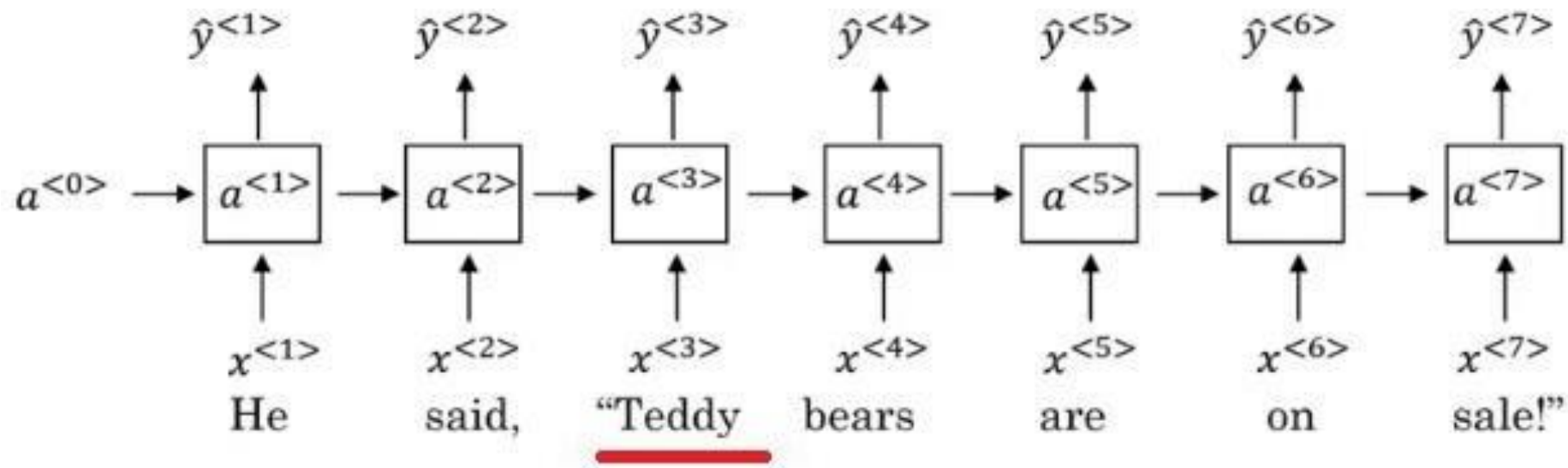
fruit or company?



BiRNN Motivation

He said, "Teddy bears are on sale!"

He said, "Teddy Roosevelt was a great President!"

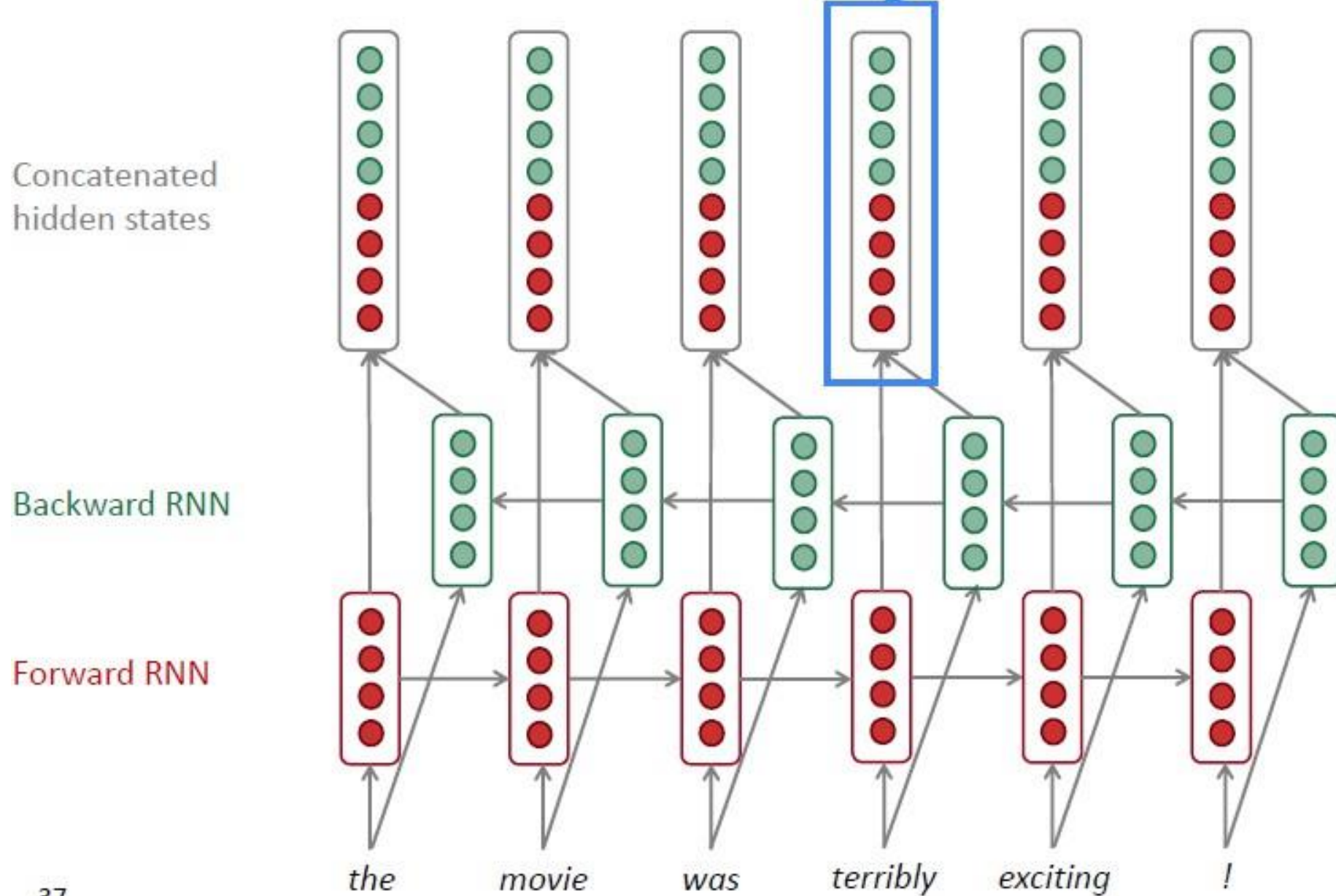


The name **Teddy** cannot be learned from **He** and **said**, but can be learned from **bears**

BiRNNs fix this issue

Bidirectional RNNs

This contextual representation of "terribly" has both left and right context!



Bidirectional RNNs

On timestep t :

This is a general notation to mean “compute one forward step of the RNN” – it could be a vanilla, LSTM or GRU computation.

Forward RNN $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

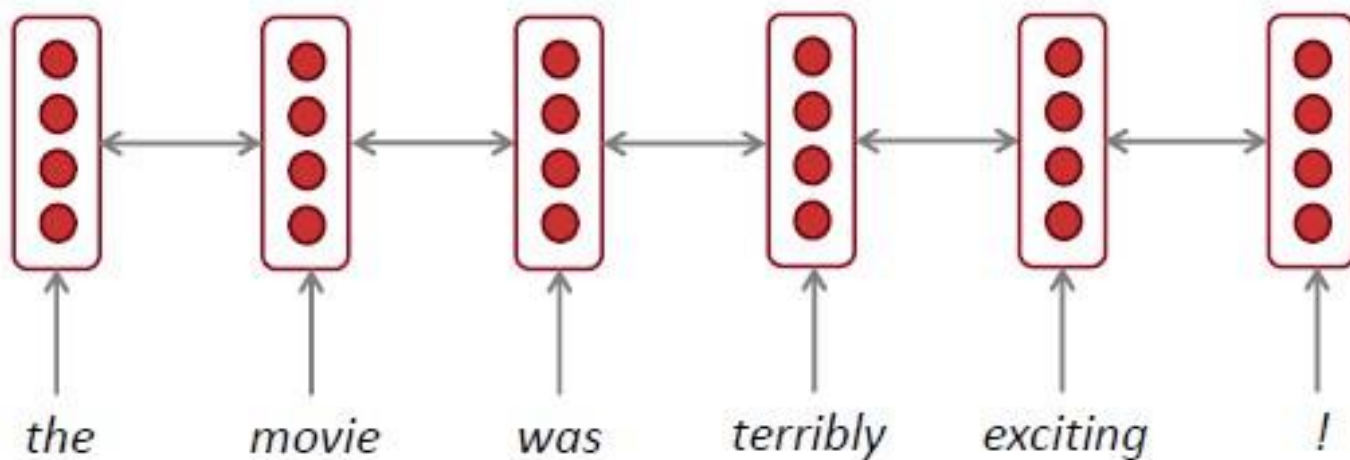
Backward RNN $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

Generally, these two RNNs have separate weights

Concatenated hidden states $\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$

We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNNs: simplified diagram



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states.

Bidirectional RNNs

- Note: bidirectional RNNs are only applicable if you have access to the entire input sequence.
- They are **not** applicable to Language Modeling, because in LM you *only* have left context available.
- If you do have entire input sequence (e.g. any kind of encoding), bidirectionality is powerful (you should use it by default).

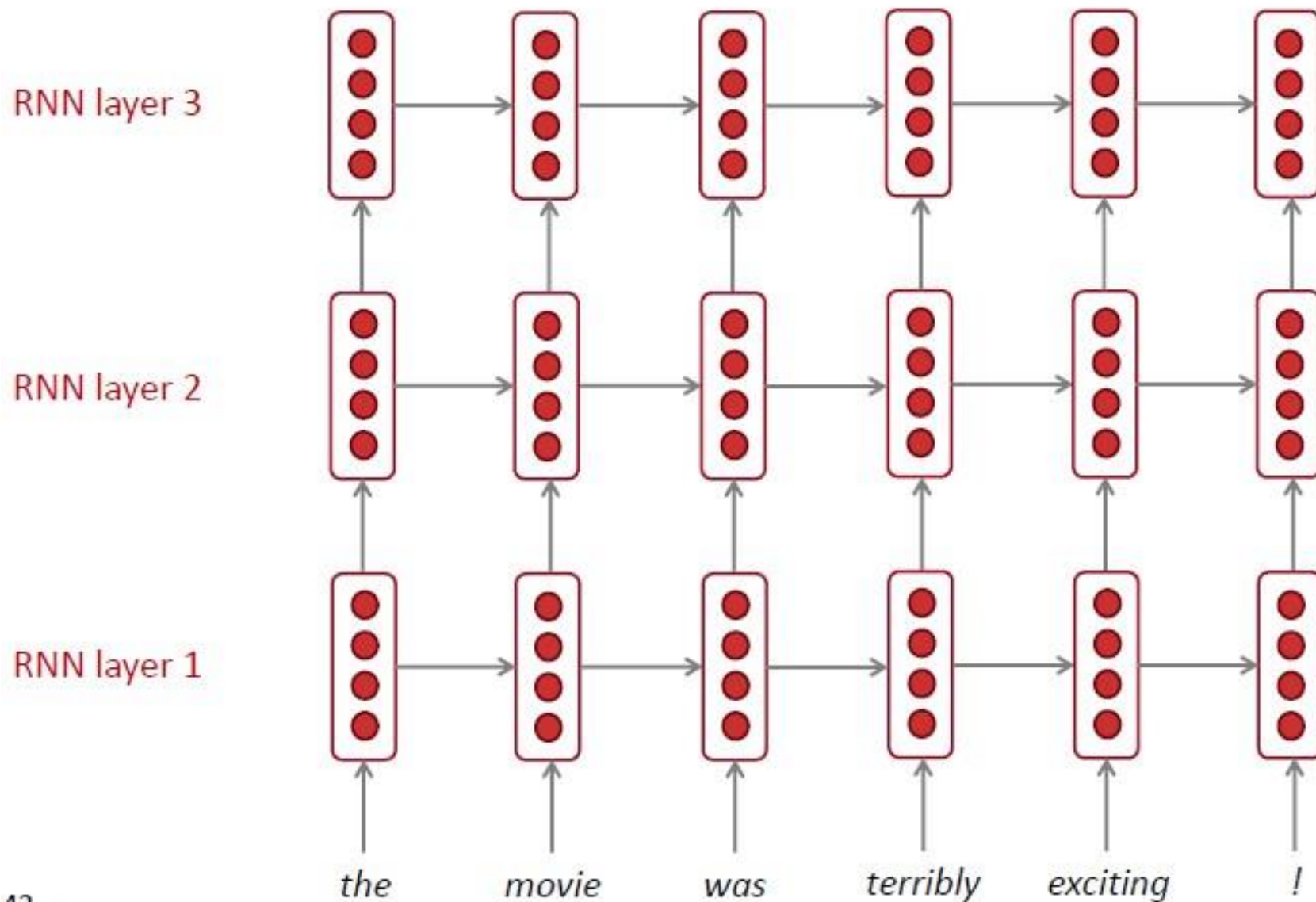
Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by *applying multiple RNNs* – this is a multi-layer RNN.
- This allows the network to compute *more complex representations*
 - The *lower RNNs* should compute *lower-level features* and the *higher RNNs* should compute *higher-level features*.
- Multi-layer RNNs are also called *stacked RNNs*.

Multi-layer RNNs

The hidden states from RNN layer i are the inputs to RNN layer $i+1$

Real-World Example: Google's Speech Recognition System

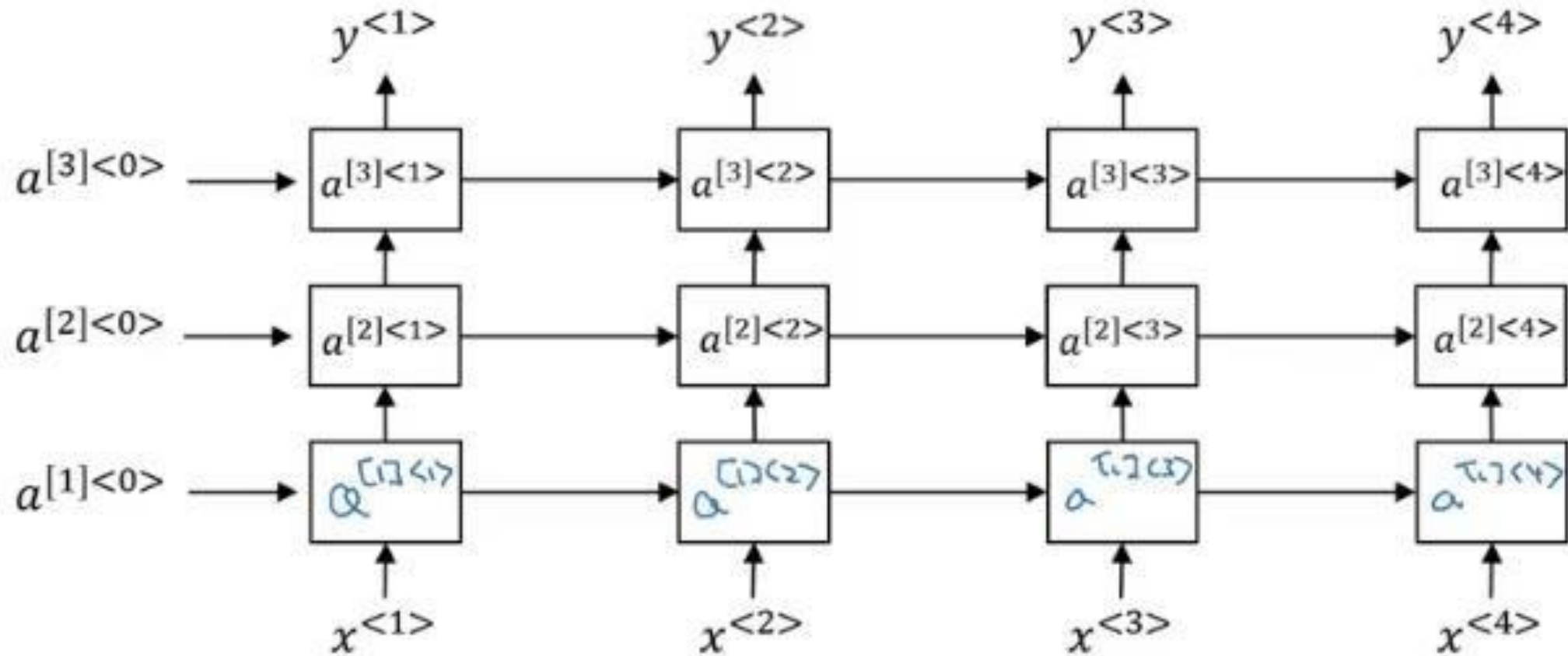


Layer 3 combines those into even higher-level structure — like recognizing whole words or common short phrases.

Layer 2 combines those into more abstract units — like recognizing partial syllables or common sound sequences.

Layer 1 learns simple, local acoustic features — like distinguishing basic sound frequencies or short phoneme boundaries.

Deep RNN



Multi-layer RNNs in practice

- High-performing RNNs are often multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For example: In a 2017 paper, Britz et al find that for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
 - However, skip-connections/dense-connections are needed to train deeper RNNs (e.g. 8 layers)
- Transformer-based networks (e.g. BERT) can be up to 24 layers
 - You will learn about Transformers later; they have a lot of skipping-like connections

Reading

- Chapter 9, Speech and Language Processing. Daniel Jurafsky & James H. Martin. Third edition

<https://web.stanford.edu/~jurafsky/slp3/9.pdf>